Introduction to Programming in Python

Course Overview

This 5-week course introduces fundamental programming concepts using Python, designed for beginners with no/limited programming experience. The course prepares students for future work with data analysis tools like pandas by establishing core programming skills.

Format: 5 weekly meetups (90 minutes each) with guided self-study and assignments between sessions

Week 1: Python Basics & Environment Setup

Meetup Topics:

- Course introduction and expectations
- Setting up Python environment (Anaconda distribution)
- Introduction to Jupyter Notebooks
- Variables, data types, and basic operations
- Getting user input and displaying output

In-Class Activities:

- Guided installation and setup
- First Jupyter Notebook creation
- Interactive coding exercises with basic operations
- Simple programs using input/output

Homework Assignment:

1. Complete Python Basics Notebook:
    a. Variable assignment exercises
    b. Type conversion challenges
    c. Basic operations with different data types
    d. Simple input/output program
2. Self-study reading: Control flow concepts
3. Install and configure all required software

Self-Study Materials:

- "Python for Data Analysis" by Wes McKinney: Chapter 2 (Python Language Basics) and Section 3.1 (Data Structures and Sequences)
- LM Notebook: "Python Basics & Environment Setup"
- Video tutorials on Python installation
- Documentation on Jupyter Notebook usage
- Practice problems with solutions

Week 2: Control Flow & Boolean Logic

Meetup Topics:

- Review of Week 1 concepts
- Boolean expressions and operators
- Conditional statements (if, elif, else)
- Loops: for and while
- Loop control (break, continue)
- Nested structures

In-Class Activities:

- Boolean logic exercises
- Implementing decision trees with conditionals
- Loop pattern exercises (counting, accumulation)
- Debugging common control flow errors

Homework Assignment:

1. Complete Control Flow Notebook:
    a. Boolean logic problems
    b. Decision structure implementations
    c. Loop implementation challenges (sum of numbers, patterns)
    d. Combined control flow program
2. Begin thinking about final project ideas
3. Self-study reading: Data structures

Self-Study Materials:

- "Python for Data Analysis" by Wes McKinney: Chapter 2.3 (Control Flow) and 2.4 (Functions)
- LM Notebook: "Control Flow & Boolean Logic"
- Interactive tutorial on boolean logic
- Practice problems involving conditionals
- Loop pattern examples
- Common algorithm implementations

Week 3: Data Structures (Lists, Dictionaries, Sets)

Meetup Topics:

- Review of Week 2 concepts
- Lists: creation, indexing, slicing, methods
- Dictionaries: key-value pairs, methods
- Sets: unique collections, operations
- List, dictionary, and set comprehensions
- Choosing appropriate data structures

In-Class Activities:

- Data structure manipulation exercises
- Converting between data structures
- Comprehension syntax practice
- Data modeling scenarios

Homework Assignment:

1. Complete Data Structures Notebook:
    a. List processing exercises
    b. Dictionary mapping implementations
    c. Set operations practice
    d. Comprehension conversion challenges
2. Submit final project proposal (1 paragraph description)
3. Self-study reading: Functions and file I/O

Self-Study Materials:

- "Python for Data Analysis" by Wes McKinney: Chapter 3 (Built-in Data Structures, Functions, and Files) - focusing on sections 3.1 and 3.2
- LM Notebook: "Data Structures & Comprehensions"
- Documentation on Python data structures
- Video tutorials on comprehensions
- Practice problems for each data structure
- Example use cases

Week 4: Functions, File I/O & Introduction to NumPy

Meetup Topics:

- Review of Week 3 concepts
- Function definition and parameters
- Return values and scope
- Reading/writing text files
- Working with CSV data
- Introduction to NumPy arrays and vectorized operations

In-Class Activities:

- Function implementation exercises
- File reading/writing practice
- CSV data processing
- Basic NumPy operations
- Project consultation and support

Homework Assignment:

1. Complete Functions and Files Notebook:
    a. Creating utility functions
    b. Reading and processing text files
    c. CSV data manipulation
    d. Basic NumPy array operations
2. Continue working on final project (should be ~50% complete)
3. Self-study: Advanced topics and project-specific resources

Self-Study Materials:

- "Python for Data Analysis" by Wes McKinney: Chapter 3.3 (Functions), 3.4 (Files and the Operating System), and Chapter 4 (NumPy Basics)
- LM Notebook: "Functions, File I/O & NumPy Introduction"
- Documentation on Python functions
- File handling examples
- NumPy quickstart guide
- Project-relevant tutorials

Week 5: Project Presentations & Advanced Concepts

Meetup Topics:

- Final project presentations
- Review of core concepts
- Common challenges and solutions
- Introduction to additional libraries
- Next steps in Python programming
- Course wrap-up

In-Class Activities:

- Project demonstrations (5-7 minutes per student)
- Peer feedback sessions
- Q&A on advanced concepts
- Discussion of future learning paths

Final Project Due at Start of Week 5

Self-Study Materials:

- "Python for Data Analysis" by Wes McKinney: Review of previous chapters and introduction to Chapter 5 (Getting Started with pandas)
- LM Notebook: "Next Steps & Advanced Concepts"

Final Project Guidelines

Project Description

Develop a data processing application using Python that demonstrates your understanding of the concepts covered in this course. The project should involve data input, processing, and output, with results displayed effectively.

Requirements

1. **Jupyter Notebook Format:** The entire project must be contained in a well-organized Jupyter Notebook with:
    a. Clear markdown sections explaining your approach
    b. Well-commented code
    c. Output results and visualizations

2. **Technical Requirements:**
    a. Use at least 3 different data types
    b. Implement control flow (conditionals and loops)
    c. Use at least 2 different data structures (lists, dictionaries, sets)
    d. Create at least 2 custom functions
    e. Read data from and/or write data to files
    f. Use list/dictionary comprehensions where appropriate
    g. Implement basic NumPy operations

3. **Project Components:**
    a. Data sourcing (can be provided files, web sources, or self-generated)
    b. Data cleaning/preprocessing
    c. Data analysis or transformation
    d. Results presentation (text output, simple visualization, etc.)
    e. Documentation of process and findings

Project Timeline

- **Week 3:** Submit project proposal (1 paragraph description)
- **Week 4:** Project check-in (discuss progress and challenges)
- **Week 5:** Final project submission and presentation

Evaluation Criteria

- **Functionality (40%):** Does the code work as intended? Does it solve the stated problem?
- **Code Quality (30%):** Is the code well-organized, properly commented, and efficiently written?
- **Concept Application (20%):** Does the project demonstrate understanding of course concepts?
- **Documentation (10%):** Is the process well-documented with clear explanations?

Sample Project Ideas (from last year's students)

1. **RNA Concentration Calculator:** Create a program that calculates RNA concentration based on RNA sequence and absorbance values. The program should process nucleotide sequences, apply relevant formulas for concentration calculation, and output concentration values for research purposes.
2. **Shümom Writing System Translator:** Develop a tool that allows users to communicate in the shümom writing system on social networks. The program should enable translation between standard text and shümom, support the creation of literary works in this writing system, and provide an intuitive interface for users unfamiliar with shümom.
3. **FISH Analysis Simulator:** Simulate FISH Analysis and detect genetic abnormalities such as deletions, amplifications and translocations. The program analyzes FISH data (red and green probe signals) and provide a diagnosis based on signal patterns. This project simulates how FISH is used to detect genetic rearrangements or imbalances.

Assessment Criteria

To successfully complete this course, you will need to:

1. **Attend all 5 weekly meetups** (20% of final grade)
2. **Complete all weekly homework assignments** (40% of final grade)

3. **Deliver and present a final project** (40% of final grade)

Required Materials

- Personal computer with internet access
- Python 3.x
- Jupyter Notebook
- VS Code
- "Python for Data Analysis" by Wes McKinney (2nd or 3rd edition)
- Course LM Notebooks

This course curriculum focuses on building fundamental Python skills with an emphasis on practical application, particularly for data processing tasks that will prepare participants for the DAF (data analysis fundamentals) module where we will work with pandas and more advanced data analysis tools.